

AI Chat SaaS Backend Architecture

This document is an onboarding map for the backend of this project. The product is a multi-tenant site-support chatbot platform: each customer can install a chat widget on their own site, visitors can ask questions, the system can answer from site knowledge when allowed, and human admins/agents can take over conversations from the management panel.

1. Big Picture

The repository has four main parts:

- `backend/`: plain PHP API backend, using PDO and MySQL.
- `frontend/`: Next.js management panel for super admins, customer admins, and agents.
- `widget/`: embeddable JavaScript chat widget for customer websites.
- `test-shop/`: a simple test site for trying the widget.

The backend is not a framework application. Each API endpoint is a standalone PHP file under `backend/api/...`, and common behavior is shared through files in `backend/includes/...`

Core backend responsibilities:

- Authenticate panel users with JWT.
- Separate data by tenant/customer and site.
- Serve public widget APIs by `site_key`.
- Store visitors, conversations, messages, attachments, and typing/presence state.
- Let human agents/admins reply to conversations.
- Generate AI reply suggestions for agents.
- Optionally auto-reply from AI when support is offline.
- Crawl customer websites into a local knowledge base.
- Track unanswered questions for knowledge improvement.
- Enforce plan limits, upload checks, CORS, origin checks, and rate limits.

2. Runtime And Configuration

Important files:

- `backend/config/app.php`: loads `.env`, sets app config, JWT settings, URLs, and timezone.
- `backend/config/database.php`: creates the global `$pdo` MySQL connection.
- `backend/.env.example`: documents required environment variables.
- `backend/public/index.php`: simple backend health response.

Important environment variables:

- `APP_ENV`, `APP_DEBUG`, `APP_TIMEZONE`
- `FRONTEND_URL`, `API_URL`, `WIDGET_SCRIPT_URL`
- `DB_HOST`, `DB_NAME`, `DB_USER`, `DB_PASS`
- `JWT_SECRET`, `JWT_EXPIRATION_SECONDS`, `JWT_MAX_TTL_SECONDS`, `JWT_ISSUER`, `JWT_AUDIENCE`
- `OPENAI_API_KEY`, `OPENAI_MODEL`

- `PANEL_ALLOWED_ORIGINS`
- `WIDGET_ALLOWED_ORIGINS`, `WIDGET_ALLOW_EMPTY_ORIGIN`
- `UPLOAD_MAX_BYTES`, `UPLOAD_MAX_IMAGE_PIXELS`, `UPLOAD_STORAGE_ROOT`, `UPLOAD_PUBLIC_URL`

Current AI answer generation in the active endpoints is mostly local/extractive. The OpenAI variables exist, and there is a backup AI suggestion endpoint, but the active `ai-answer-engine.php` searches local knowledge tables rather than calling an external LLM.

3. Roles And Access Model

The backend uses these roles:

- `super_admin`: platform owner. Can manage plans, tenants/customers, sites, users, announcements, and platform stats.
- `customer_admin`: customer-side administrator. Can manage their tenant's sites, agents, widget settings, AI settings, knowledge, crawl sources, quick replies, and reports.
- `agent`: support user. Can view and reply to conversations for sites they are assigned to.

Authentication is handled by:

- `backend/api/auth/login.php`
- `backend/api/auth/me.php`
- `backend/api/auth/change-password.php`
- `backend/api/auth/logout-all.php`
- `backend/includes/auth.php`
- `backend/includes/jwt.php`

Panel endpoints expect `Authorization: Bearer <token>`.

JWT details:

- The backend signs tokens with HS256.
- Tokens include `sub`, `tenant_id`, `email`, `role`, `token_version`, `jti`, `iat`, `nbft`, `exp`, `iss`, and `aud`.
- `token_version` is checked against the database, so password resets or logout-all can revoke old tokens.
- Non-super-admin users must belong to an active tenant.

Site access is enforced in `backend/includes/site-access.php`:

- Super admins can access all sites.
- Customer admins can access sites under their tenant.
- Agents can access only sites listed in `agent_site_access`.

4. Multi-Tenant Domain Model

The full base database schema is not included in this repository. Only the AI migration exists:

- `backend/database/migrations/2026_07_07_ai_knowledge_tables.sql`

The core tables are inferred from API usage:

- `plans`: plan name, site/agent/conversation limits, AI feature flags, active status.

- **tenants**: customer accounts, owner info, plan assignment, status.
- **sites**: customer websites, domain, **site_key**, widget branding, **ai_mode**, active status.
- **users**: panel users, roles, password hashes, token versions, presence fields.
- **agent_site_access**: links agents/admins to sites.
- **visitors**: public website visitors, browser ID, contact fields, IP/user agent.
- **conversations**: one support thread per visitor/site, status, assigned agent, source page, last message time.
- **messages**: visitor, agent, and AI messages.
- **message_attachments**: uploaded files attached to messages.
- **quick_replies**: reusable support replies.
- **knowledge_sources**: manually added customer knowledge.
- **ai_suggestions**: AI draft replies for agents.
- **api_rate_limits**: rate limiting records.
- **announcements**: platform/customer announcements.

AI-specific tables from the included migration:

- **ai_site_settings**: per-site AI toggles and thresholds.
- **ai_crawl_sources**: URLs, path prefixes, or sitemaps to crawl.
- **ai_crawl_runs**: crawl execution status and counters.
- **ai_pages**: fetched pages and cleaned text.
- **ai_content_chunks**: searchable chunks derived from pages.
- **ai_terms**: extracted terms used for scoring boosts.
- **ai_generated_questions**: template-generated questions per chunk.
- **ai_unanswered_questions**: visitor questions that were not confidently answered.
- **ai_answer_logs**: every AI suggestion/auto-reply/fallback/no-answer event.

5. Main API Groups

Auth

Path: **backend/api/auth/***

- **login.php**: validates email/password, rate limits attempts, returns JWT.
- **me.php**: returns current authenticated user.
- **change-password.php**: changes current user's password and increments token version.
- **logout-all.php**: revokes existing tokens by incrementing token version.

Widget Public API

Path: **backend/api/widget/***

These endpoints are public but guarded by **site_key**, site status, tenant status, widget origin validation, and rate limits.

- **config.php**: returns widget branding, welcome message, **ai_mode**, and whether support is online.
- **visitor-start.php**: creates or updates a visitor by **browser_id**.
- **conversation-start.php**: returns an active conversation or creates a new one, enforcing monthly plan limits.

- `message-send.php`: stores visitor text messages.
- `attachment-send.php`: stores visitor file attachments.
- `messages-list.php`: returns conversation messages and attachments.
- `typing-status.php` and `typing-update` style endpoints: expose typing state.
- `ai-reply.php`: creates an automatic AI message when enabled and support is offline.

Important detail: `message-send.php` only stores the visitor message. Automatic AI response is separate in `widget/ai-reply.php`, which expects the visitor message ID.

Agent API

Path: `backend/api/agent/*`

Used by customer admins and agents inside the panel.

- `conversations-list.php`: lists conversations visible to the user.
- `conversation-show.php`: loads one conversation and messages.
- `message-send.php`: sends a human reply and assigns the conversation if unassigned.
- `attachment-send.php`: uploads an agent attachment.
- `conversation-assign.php`: assigns a conversation to an agent.
- `conversation-status-update.php`: changes conversation status.
- `conversation-close.php`: closes a conversation.
- `presence-update.php`, `presence-status.php`: support online/offline presence.
- `typing-update.php`: typing state.
- `quick-replies-list.php`: reusable replies.
- `assignable-agents-list.php`: agents available for assignment.
- `ai-suggestion-generate.php`: generates or updates a pending AI suggestion from the latest visitor message.
- `ai-suggestions-list.php`: lists suggestions.
- `ai-suggestion-accept.php`: marks a suggestion accepted, edited, or rejected.
- `ai-suggestion-mark-used.php`: records use of a suggestion.

Customer Admin API

Path: `backend/api/customer/*`

Used by `customer_admin`.

- Site management: `sites-list.php`, `widget-settings-update.php`
- Team management: `team-list.php`, `team-create.php`
- Plan usage: `plan-usage.php`
- Reports: `reports-summary.php`
- Quick replies: `quick-replies-list.php`, `quick-replies-create.php`, `quick-replies-delete.php`
- Announcements: `announcements-list.php`, `announcement-read.php`, `announcement-dismiss.php`
- Manual knowledge: `knowledge-list.php`, `knowledge-create.php`, `knowledge-delete.php`
- AI settings: `ai-settings.php`
- AI search test and overview: `ai-search-test.php`, `ai-overview.php`
- Crawl sources: `ai-crawl-sources-list.php`, `ai-crawl-source-create.php`, `ai-crawl-source-delete.php`

- Crawl execution: `ai-crawl-start.php`
- Knowledge source management: `ai-knowledge-sources-list.php`, `ai-knowledge-source-create.php`, `ai-knowledge-source-update.php`
- Generated questions: `ai-generated-questions-list.php`, `ai-generated-question-update.php`
- Unanswered queue: `ai-unanswered-list.php`, `ai-unanswered-update-status.php`, `ai-unanswered-add-to-knowledge.php`

Super Admin API

Path: `backend/api/super-admin/*`

Used by `super_admin`.

- Dashboard: `dashboard-stats.php`
- Customers/tenants: `tenants-list.php`, `tenants-options.php`, `customer-create.php`, `customer-show.php`, `customer-status-update.php`, `customer-plan-update.php`
- Plans: `plans-list.php`, `plan-create.php`, `plan-update.php`, `plan-toggle-status.php`
- Sites: `sites-list.php`, `site-settings-update.php`, `site-status-update.php`
- Users: `user-status-update.php`, `user-password-reset.php`
- Announcements: `announcements-list.php`, `announcement-create.php`, `announcement-update.php`, `announcement-delete.php`, `announcement-image-upload.php`

6. Chat And Handoff Flow

Typical visitor flow:

1. Widget loads on a customer site with a `site_key`.
2. Widget calls `widget/config.php` to load branding, welcome text, AI mode, and support-online status.
3. Widget calls `widget/visitor-start.php` with `site_key` and `browser_id`.
4. Widget calls `widget/conversation-start.php`.
5. Visitor sends a message through `widget/message-send.php`.
6. The widget or frontend can call `widget/ai-reply.php` with the created `message_id`.
7. Agents see the conversation in the panel through `agent/conversations-list.php` and `agent/conversation-show.php`.
8. A human replies through `agent/message-send.php`; the conversation is assigned if it had no assignee.

Handoff behavior:

- If support is online, `widget/ai-reply.php` skips auto-reply with reason `support_online`.
- If support is offline and auto-reply is enabled, AI can insert a message with `sender_type = 'ai'`.
- If AI cannot answer confidently, it inserts the configured fallback message and logs the question as unanswered.
- Human agents can still see and continue the conversation.

Conversation statuses seen in code:

- `new`
- `open`
- `in_progress`
- `waiting_customer`

- `follow_up`
- `pending`
- `closed`

Message sender types:

- `visitor`
- `agent`
- `ai`

7. AI Architecture

The AI system has two modes:

- Agent suggestions: drafts a response for a human support user to review.
- Widget auto-reply: sends a response directly to the visitor when support is offline and confidence is high enough.

Site-level AI controls:

- `sites.ai_mode`: `off`, `assistant`, or `semi_auto`.
- `ai_site_settings.assistant_enabled`
- `ai_site_settings.auto_reply_enabled`
- `ai_site_settings.crawl_enabled`
- `ai_site_settings.min_auto_reply_score`
- `ai_site_settings.min_suggestion_score`
- `ai_site_settings.max_pages_per_crawl`
- `ai_site_settings.max_depth`
- `ai_site_settings.fallback_message`

Plan-level AI controls:

- `plans.ai_suggestions_enabled`
- `plans.ai_auto_reply_enabled`
- `plans.knowledge_base_enabled`

Important implementation files:

- `backend/includes/ai-crawler.php`
- `backend/includes/ai-answer-engine.php`
- `backend/includes/ai-helpers.php`
- `backend/includes/knowledge-search.php`
- `backend/api/customer/ai-crawl-start.php`
- `backend/api/agent/ai-suggestion-generate.php`
- `backend/api/widget/ai-reply.php`

Knowledge Ingestion

Customer admins configure crawl sources, then start a crawl:

1. `ai-crawl-start.php` loads active sources for the site.

2. Sources can be:

- `url`
- `path_prefix`
- `sitemap`

3. The crawler fetches HTML with `curl` or `file_get_contents`.

4. It extracts title, meta description, H1, readable text, and links.

5. It validates that discovered URLs belong to the site's domain.

6. It stores cleaned page content in `ai_pages`.

7. It splits page text into chunks in `ai_content_chunks`.

8. It extracts terms into `ai_terms`.

9. It creates template questions in `ai_generated_questions`.

10. It records the crawl summary in `ai_crawl_runs`.

The crawler is synchronous. A crawl request can do all fetch, extraction, storage, and response work inside the HTTP request.

Answer Search

`ai_find_best_answer()` in `ai-answer-engine.php` is the main answer function.

It searches three knowledge sources:

- Manual `knowledge_sources`
- Crawled `ai_content_chunks`
- Template `ai_generated_questions`

Scoring is local and rule-based:

- Normalize the visitor question.
- Extract non-stopword tokens.
- Detect rough intent/category from keywords.
- Search candidates with SQL `LIKE` conditions.
- Boost matches based on terms, intent, category, manual knowledge, FAQ type, and exact-ish question matching.
- Pick the best scored candidate.
- Build a short extractive reply from the best source text.

The answer engine returns:

- `success`
- `reply_mode`
- normalized question and tokens
- detected category/intent
- `confidence_score` from 0 to 100
- answer text
- matched source IDs
- top sources and candidates

Agent Suggestions

`backend/api/agent/ai-suggestion-generate.php`:

- Requires `customer_admin` or `agent`.
- Requires the plan feature `ai_suggestions_enabled`.
- Checks site access.
- Loads the latest visitor message in the conversation.
- Calls `ai_find_best_answer()`.
- Uses `min_suggestion_score` from `ai_site_settings`, default 45.
- Creates or updates a pending row in `ai_suggestions`.
- Logs to `ai_answer_logs`.
- If confidence is too low, stores the question in `ai_unanswered_questions`.

Widget Auto-Reply

`backend/api/widget/ai-reply.php`:

- Validates `site_key`, visitor, conversation, message, active site, active tenant, and origin.
- Skips if the conversation is closed.
- Skips if `sites.ai_mode = 'off'`.
- Skips if `ai_site_settings.assistant_enabled` is off.
- Skips if `auto_reply_enabled` is off.
- Skips if support is online in the last 2 minutes.
- Prevents duplicate auto replies for the same visitor message.
- Calls `ai_find_best_answer()`.
- Uses `min_auto_reply_score`, default 75.
- Inserts an AI message if confident enough.
- Otherwise inserts fallback text and records an unanswered question.
- Logs the result in `ai_answer_logs`.

8. Manual Knowledge And Unanswered Questions

Manual knowledge lives in `knowledge_sources` and is managed by customer endpoints:

- `knowledge-create.php`
- `knowledge-list.php`
- `knowledge-delete.php`
- `ai-knowledge-source-create.php`
- `ai-knowledge-source-update.php`
- `ai-knowledge-sources-list.php`

The AI answer engine gives manual knowledge a strong boost. That means curated FAQ/knowledge entries should usually beat raw crawled page chunks.

Unanswered questions live in `ai_unanswered_questions`.

The intended improvement loop is:

1. Visitor asks something.
2. AI score is below threshold.
3. Question is stored as unanswered.

4. Customer admin reviews it.
5. Admin can mark it reviewed/ignored or add it to knowledge.
6. Future answer quality improves.

9. Plans, Limits, And Usage

Plan enforcement is in `backend/includes/plan-limits.php`.

Supported limits/features:

- Maximum sites per tenant.
- Maximum active agents/users.
- Maximum monthly conversations.
- AI suggestions feature flag.
- AI auto-reply feature flag.
- Knowledge base feature flag.

Important behavior:

- `conversation-start.php` enforces monthly conversation limits before creating a new conversation.
- `team-create.php` enforces agent limits.
- Site creation uses plan limits where the relevant helper is called.
- AI suggestion generation checks `ai_suggestions_enabled`.

10. Security And Abuse Protection

Cross-cutting security files:

- `backend/includes/security-headers.php`
- `backend/includes/cors.php`
- `backend/includes/widget-cors.php`
- `backend/includes/rate-limit.php`
- `backend/includes/upload.php`
- `backend/includes/response.php`
- `backend/includes/error-handler.php`

Security measures currently present:

- JWT authentication for panel APIs.
- Role checks on protected endpoints.
- Site access checks to prevent IDOR across tenant/site boundaries.
- Panel CORS restricted by `PANEL_ALLOWED_ORIGINS`.
- Widget origin validation against the site domain, local dev origins, or `WIDGET_ALLOWED_ORIGINS`.
- Rate limiting stored in `api_rate_limits`.
- Standard JSON response shape and production sanitization for 500 errors.
- Common security headers: nosniff, frame deny, referrer policy, permissions policy, strict CSP for APIs.
- File upload validation:
 - size limits
 - MIME and extension allowlist
 - dangerous extension blocking

- payload scans for scripts/HTML/SVG/executables
- PDF active-content checks
- image dimension checks
- randomized stored names
- `.htaccess` protection in upload directories

Important caveat: the crawler disables SSL verification in fetches. That may be acceptable for local/MVP crawling, but it is not ideal for production security.

11. Attachments

Visitor and agent attachment endpoints use `includes/upload.php`.

Allowed file types:

- JPG/JPEG
- PNG
- GIF
- WebP
- PDF

Uploads are stored under an upload root by channel and date. Public URLs are built from `UPLOAD_PUBLIC_URL` or the current backend URL.

Attachments are linked to messages through `message_attachments`.

12. Presence, Typing, And Online Support

The backend uses `users.availability_status` and `users.last_seen_at` to decide whether support is online.

Support is considered online when:

- user is active
- role is `customer_admin` or `agent`
- `availability_status = 'online'`
- `last_seen_at >= NOW() - INTERVAL 2 MINUTE`

This matters because widget auto-reply skips if support is online. Presence is updated through agent endpoints such as `presence-update.php`; widget config exposes the online flag to visitors.

13. Widget Settings

Widget settings are stored mainly on `sites`:

- `brand_name`
- `brand_color`
- `logo_url`
- `welcome_message`
- `ai_mode`
- `site_key`

- `domain`
- `is_active`

`customer/widget-settings-update.php` lets customer admins edit branding and AI mode.

`widget/config.php` returns the public settings used by the embedded widget.

14. Super Admin Customer Creation Flow

`super-admin/customer-create.php` creates a working customer in one transaction:

1. Validate plan, admin email, domain, and password.
2. Insert `tenants` row.
3. Generate a `site_key`.
4. Insert initial `sites` row with `ai_mode = 'assistant'`.
5. Insert the customer admin user.
6. Insert `agent_site_access` for that admin and site.
7. Return tenant, site, admin, and install code details.

15. Reporting And Dashboard Data

Reporting is SQL-based and reads from:

- `tenants`
- `sites`
- `users`
- `visitors`
- `conversations`
- `messages`
- `plans`
- AI logs/settings depending on endpoint

Important endpoints:

- `customer/reports-summary.php`
- `customer/plan-usage.php`
- `customer/ai-overview.php`
- `super-admin/dashboard-stats.php`
- `super-admin/tenants-list.php`

16. Key Backend Files To Read First

Start with these files:

1. `backend/api/widget/config.php`
2. `backend/api/widget/visitor-start.php`
3. `backend/api/widget/conversation-start.php`
4. `backend/api/widget/message-send.php`
5. `backend/api/widget/ai-reply.php`
6. `backend/api/agent/conversation-show.php`

7. `backend/api/agent/message-send.php`
8. `backend/api/agent/ai-suggestion-generate.php`
9. `backend/includes/ai-answer-engine.php`
10. `backend/includes/ai-crawler.php`
11. `backend/includes/auth.php`
12. `backend/includes/site-access.php`
13. `backend/includes/plan-limits.php`
14. `backend/database/migrations/2026_07_07_ai_knowledge_tables.sql`

17. Known Gaps And Things To Verify

These are not necessarily bugs, but they are important onboarding notes:

- The repository does not include the base schema migration for core tables like `users`, `tenants`, `sites`, `conversations`, and `messages`.
- The active AI engine is not truly generative. It is local keyword/rule/extractive search over stored knowledge.
- The crawler is synchronous and may time out on larger crawls.
- The crawler disables SSL verification.
- Some PHP comments and Persian strings appear mojibake-encoded in this checkout, but runtime behavior may still work if database/frontend strings are handled consistently.
- Some endpoints return debug fields in non-production; production response sanitization mainly covers 500 responses through `json_response`.
- Plan feature enforcement exists, but it should be audited endpoint by endpoint before billing-sensitive launch.
- There is no centralized router. Adding an endpoint means adding a new PHP file and manually including the right helpers.

18. Mental Model For The Product

Think of the backend as three cooperating systems:

1. SaaS control plane: Super admin manages tenants, sites, plans, users, and platform announcements.
2. Human support inbox: Visitors create conversations from the widget. Agents/admins see conversations in the panel, assign them, reply, upload files, use quick replies, and close conversations.
3. AI knowledge assistant: Customer admins add/crawl knowledge. The backend searches that knowledge to draft replies for agents or auto-reply to visitors when support is offline and confidence is high enough.

The most important product rule is: AI is a helper, not the only support path. The backend is explicitly built so a human admin or agent can answer, override, continue, and improve the knowledge base when AI is not confident.